LINEAR REGRESSION

Simple & Multiple Linear Regression

Elsa Ferreira Gomes

2026 / 2027

Source: James, Witten, Hastie & Tibshirani — An Introduction to Statistical Learning with Python (Chp. 3)

How to find an approximated function

Regression is a simple approach to supervised learning, where the dependence of Y on x_1, x_2, ..., x_n is 'linear'.

Consider the simple linear regression — using a single predictor X:

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

β_0 (intercept) and β_1 (slope) are unknown constants (coefficients), and ε is the error.

Estimating β_0 and β_1, we get the prediction rule:

$$\hat{y} = f(X) = \hat{\beta}_0 + \hat{\beta}_1 X$$

Step 1 — Visualize the data

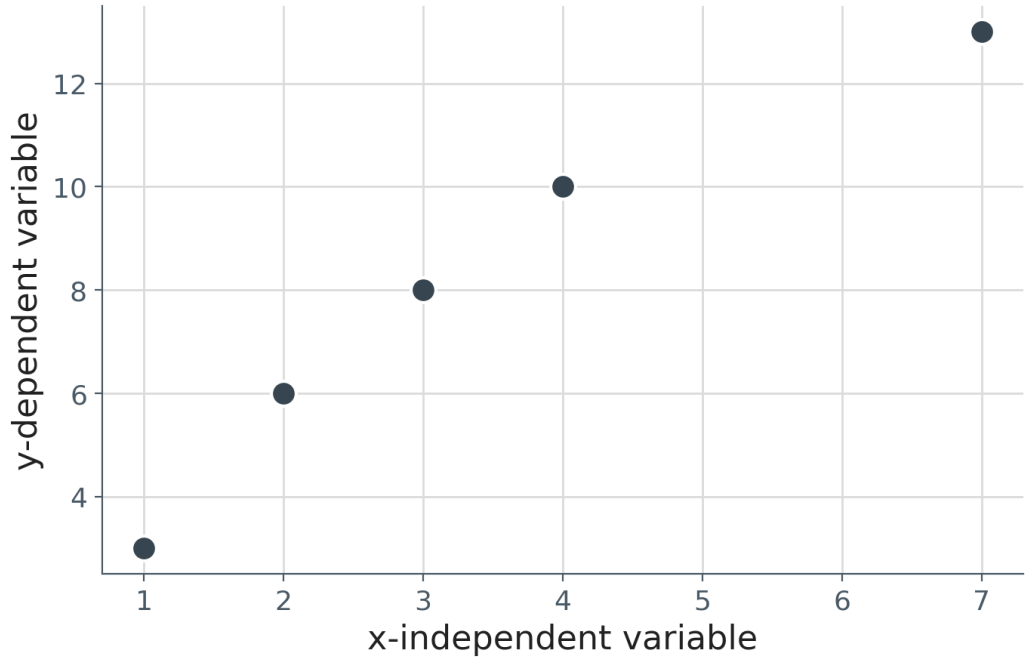
Before building a Linear Regression model, plot the data points to see the relationship between x and y.

PYTHON

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

X = [1, 2, 3, 4, 7]
Y = [3, 6, 8, 10, 13]

plt.scatter(X, Y)
plt.xlabel('x-independent variable')
plt.ylabel('y-dependent variable')
plt.title('Linear Regression example')
plt.grid(True)
plt.show()
```



Step 2 — Build the model

Using the scikit-learn library:

PYTHON

```
import sklearn.linear_model as skl_lm

model = skl_lm.LinearRegression()

X2 = [[x] for x in X]
Y = [y for y in Y]

model.fit(X2, Y)
```

model.fit(X2, Y) finds the β_0 and β_1 that best fit the 5 data points, using the least-squares method (explained shortly).

The fitted coefficients

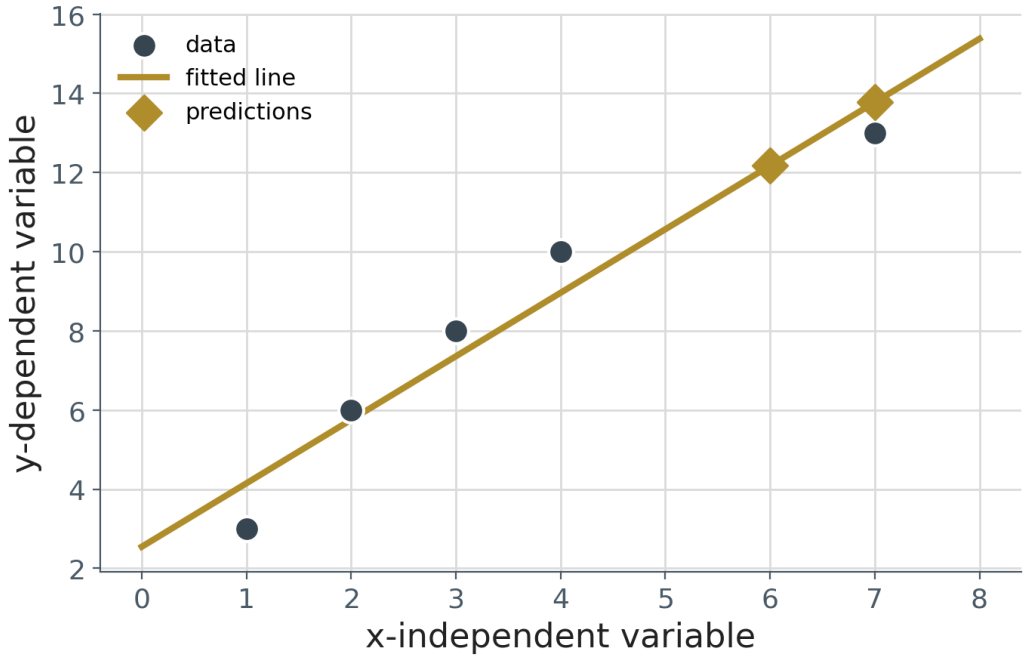
PYTHON

```
b1 = model.coef_[0]
b0 = model.intercept_

print("Slope (b1):", b1)
print("Y-intercept (b0):", b0)
```

RESULT

```
Slope (b1):      1.6038
Y-intercept (b0): 2.5472
```



Step 3 — Make predictions

For new values of X, not seen during training:

PYTHON

```
new_X = [[6], [7]]
predictions = model.predict(new_X)

print(f"Prediction for X=6: {predictions[0]:.4f}")
print("Prediction for X=7:", predictions[1])
```

RESULT

```
Prediction for X=6: 12.1698
Prediction for X=7: 13.7736
```

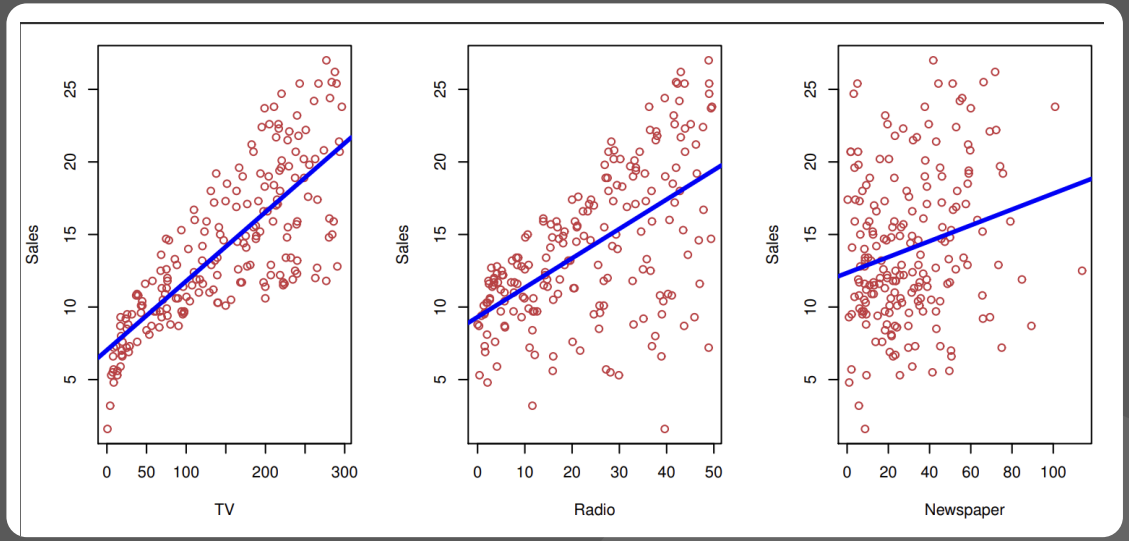
The orange diamonds on the earlier chart mark exactly these two predicted points, sitting on the fitted line.

CASE STUDY

Example: Advertising Data

The dataset captures the **sales revenue** generated with respect to advertisement costs across TV, radio and newspaper.

- Is there a relationship between advertising budget and sales?
- Is the relationship linear?
- Which media contribute to sales?



THEORY

Estimating the coefficients by least squares

$\hat{y}$ is the prediction of Y on the basis of X = x. The hat symbol denotes an *estimated* value:

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$$

How do we measure how close the predicted $\hat{y}$ are to the actual y?

The difference $e_i = \hat{y}_i - y_i$ is a **residual** (or error).

The best possible fit has $e_i = 0$ for all i — in practice, we look for the fit that makes these residuals as small as possible, overall.

THEORY

RSS — Residual Sum of Squares

*The **loss function** $\mathcal{L}$ that the least-squares method minimizes:*

$$RSS = \Sigma\, e_i^2 = \Sigma\, (y_i - (\beta_0 + \beta_1\, x_i))^2$$

***Least squares finds the β that minimize RSS** given the data — in closed form:*

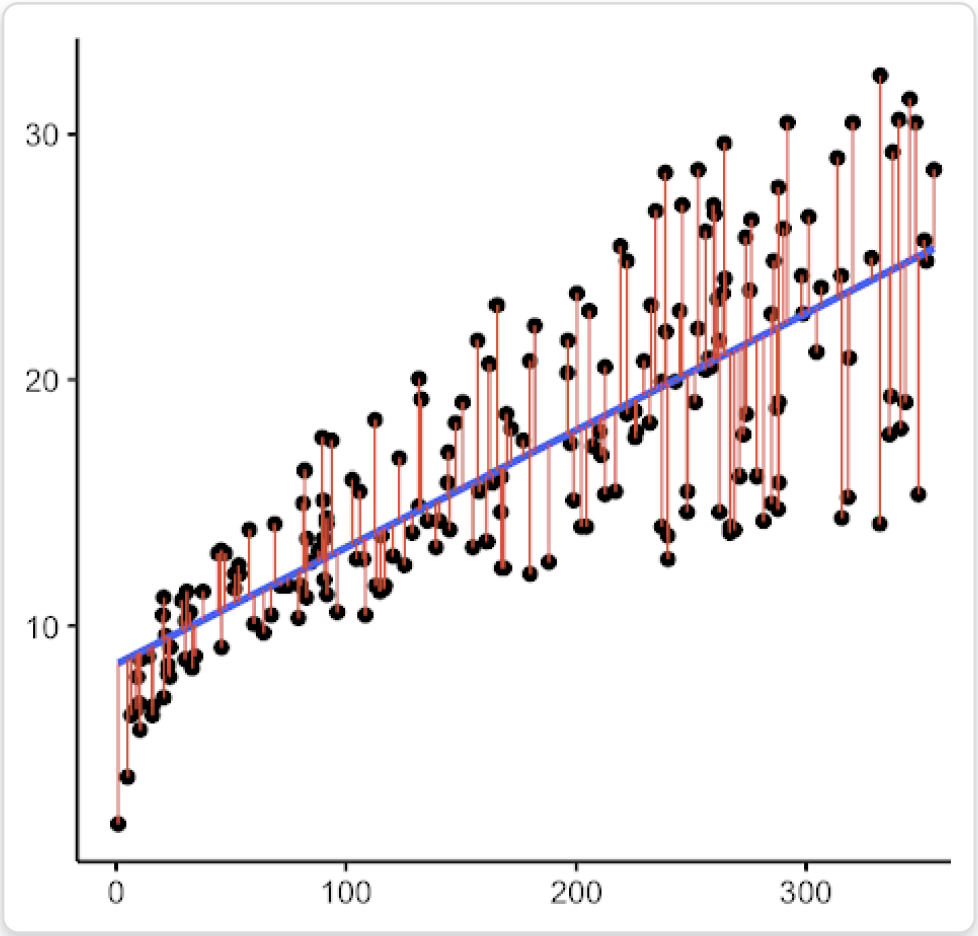
$$\beta_1 = \Sigma(x_i - \bar{x})(y_i - \bar{y}) \,/\, \Sigma(x_i - \bar{x})^2$$

$$\beta_0 = \bar{y} - \beta_1\, \bar{x}$$

where $\bar{x}$ and $\bar{y}$ are the sample means of x and y.

RSS on the advertising data

The least-squares fit for the regression of sales onto TV. The line segments represent the residuals (errors) being minimized.



THEORY

R^2 — the intrinsic quality of the model

RSS measures total error in the units of Y:

$$RSS = \Sigma\ (y_i - \hat{y}_i)^2$$

We normalize it with the Total Sum of Squares, $TSS = \Sigma(y_i - \bar{y})^2$, to get a scale-independent measure:

$$R^2 = (TSS - RSS) / TSS = 1 - RSS/TSS$$

R^2 is the proportion of variance explained by the model — a value between 0 and 1, independent of the scale of Y, so it can be compared across different models.

Loading the advertising dataset

PYTHON

```
from sklearn.preprocessing import scale
import sklearn.linear_model as skl_lm
from sklearn.metrics import (
    mean_squared_error, r2_score)

advertising = pd.read_csv(
    'Advertising.csv',
    usecols=[1,2,3,4])
advertising.info()
```

OUTPUT

```
RangeIndex: 200 entries
Data columns (total 4):
 TV          200 non-null float64
 radio       200 non-null float64
 newspaper   200 non-null float64
 sales       200 non-null float64
```

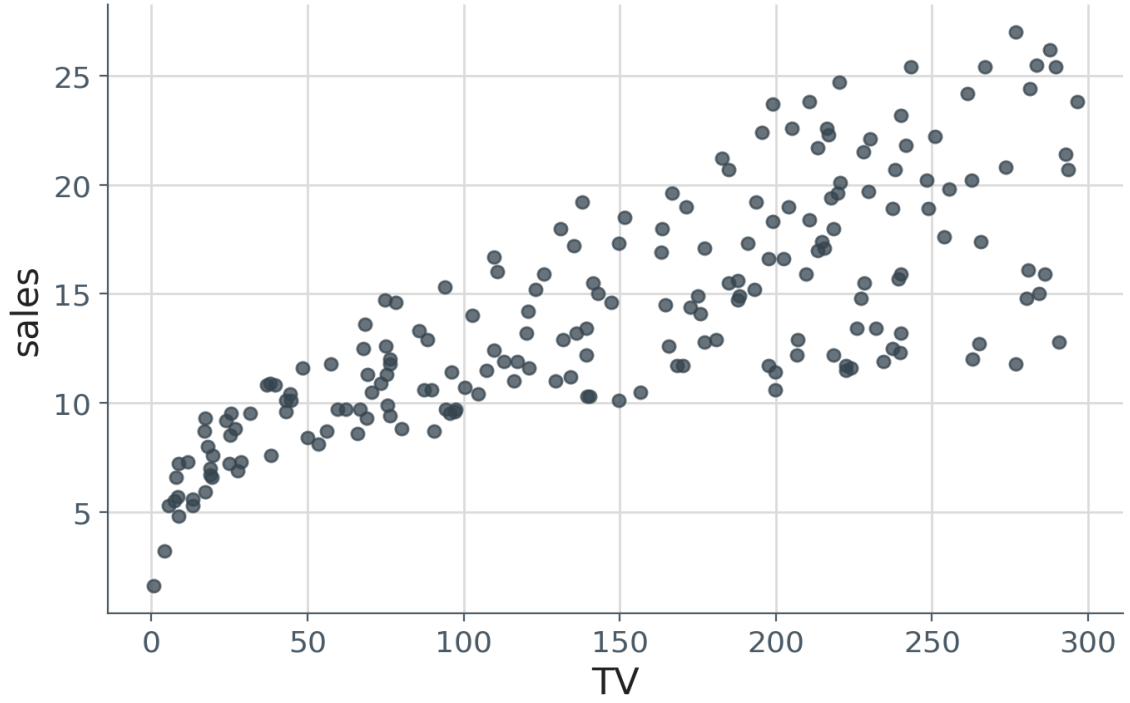
200 observations — advertising budgets (in thousands of dollars) for TV, radio and newspaper, and resulting sales (in thousands of units).

Visualizing TV vs. sales

PYTHON

```
plt.scatter(
    x="TV", y="sales",
    data=advertising)
```

There seems to be a positive relationship between TV advertising spend and sales… but is it linear?

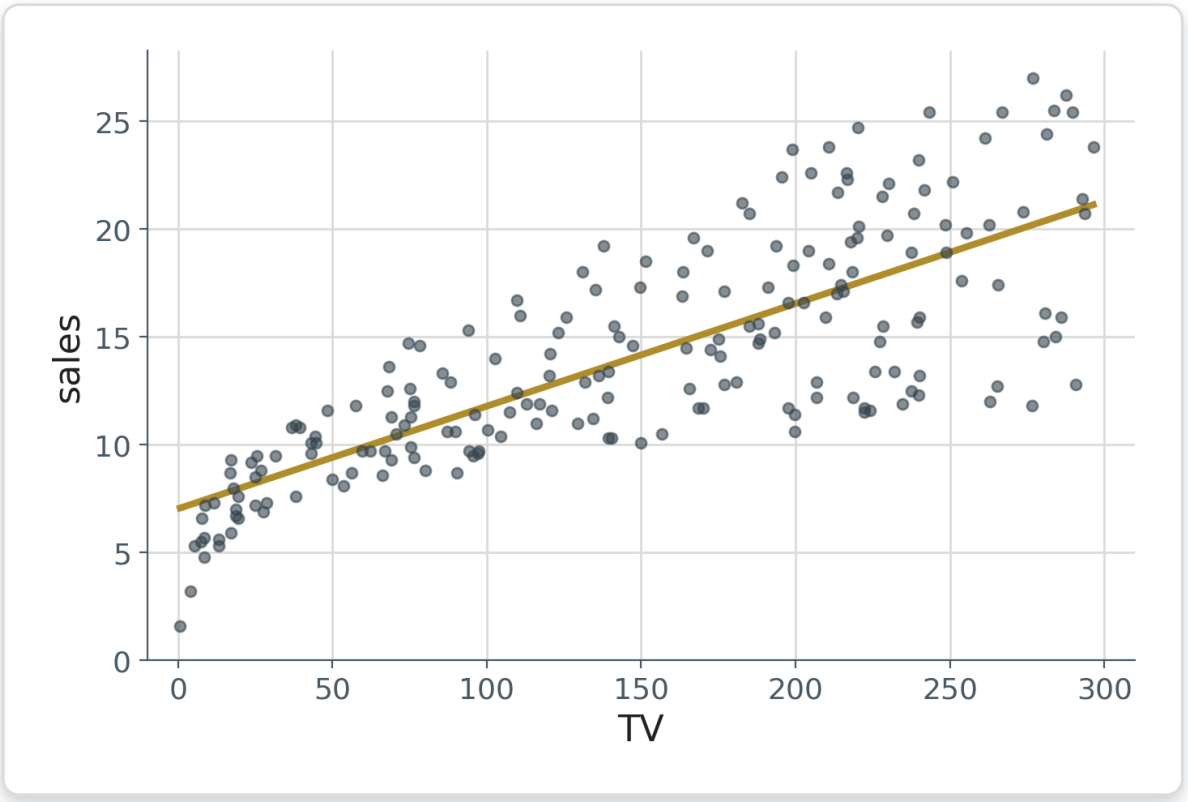


Least squares fit

Plot the data and the fitted model — consider sales = $\beta_0 + \beta_1$ TV + ε

PYTHON

```
sns.regplot(
    x="TV", y="sales",
    data=advertising,
    order=1, ci=None,
    scatter_kws={'s': 9})
plt.xlim(-10, 310)
plt.ylim(ymin=0)
```



Computing the regression coefficients

PYTHON

```
regr = skl_lm.LinearRegression()

X = scale(advertising.TV,
    with_mean=True,
    with_std=False).reshape(-1,1)
y = advertising.sales

regr.fit(X, y)
print(regr.intercept_)
print(regr.coef_)
```

RESULT

```
intercept = 14.0225
coef      =  0.0475
```

Here X was mean-centered first (with_mean=True), which changes the intercept's value but not the model's fit or slope.

Mapping the RSS across (β_0, β_1)

To find the minimum visually, compute RSS over a grid of candidate coefficient values:

PYTHON

```
B0 = np.linspace(regr.intercept_-2,
                 regr.intercept_+2, 50)
B1 = np.linspace(regr.coef_-0.02,
                 regr.coef_+0.02, 50)
xx, yy = np.meshgrid(B0, B1)
Z = np.zeros((B0.size, B1.size))

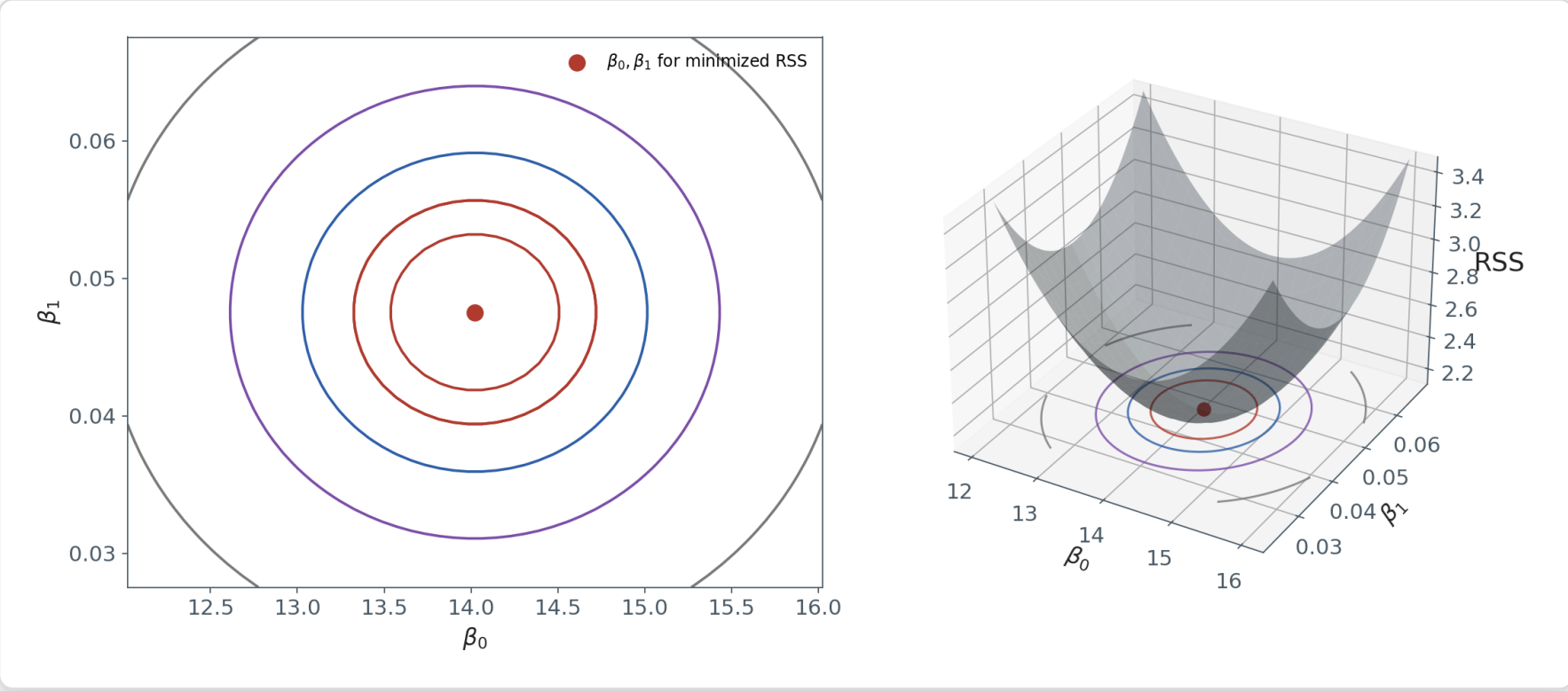
for (i,j), v in np.ndenumerate(Z):
    Z[i,j] = ((y - (xx[i,j] +
        X.ravel()*yy[i,j]))**2).sum()/1000
```

MIN RSS (÷1000)

2.1025

The RSS surface — contour & 3D

The red dot marks the least-squares estimates β_0 and β_1 — the bottom of the RSS “bowl”.



Why this matters for convergence: RSS is a quadratic (convex) function of β — the closed, nested contour rings show a single bowl-shaped minimum, with no other valleys. So least squares always converges to the same, unique solution, regardless of the starting point.

THEORY

Confidence interval

Is the relationship we found real, or could it be due to chance? We test:

H_o

There is no relationship between X and Y
($\beta_1 = 0$)

H_a

There is some relationship between X and Y
($\beta_1 \neq 0$)

We compute the t-test to obtain the p-value — a small p-value (typically < 0.05) lets us reject H_o and conclude the relationship is statistically significant.

Statsmodels OLS — sales ~ TV

PYTHON

```
import statsmodels.formula.api as smf

est = smf.ols('sales ~ TV',
              advertising).fit()
est.summary().tables[1]
```

	coef	std err	t	P>|t|
Intercept	7.0326	0.458	15.360	0.000
TV	0.0475	0.003	17.668	0.000

$R^2 = 0.6119$ — both coefficients have p-value < 0.001: the relationship between TV spend and sales is highly significant.

Cross-checking with scikit-learn

PYTHON

```
# RSS with regression coefficients
((advertising.sales - (est.params.iloc[0]
  + est.params.iloc[1]*advertising.TV)
 )**2).sum() / 1000

regr = skl_lm.LinearRegression()
X = advertising.TV.values.reshape(-1,1)
y = advertising.sales
regr.fit(X, y)

Sales_pred = regr.predict(X)
r2_score(y, Sales_pred)
```

RESULT

```
RSS/1000 = 2.1025

intercept = 7.0326
coef      = 0.0475

R² = 0.6119
```

Same numbers as statsmodels — confirms both libraries agree on the fitted model.

THEORY

Estimating parameters — the general case

RSS for the multidimensional case, where X is the N × p data matrix:

$$\mathrm{RSS}(\beta) = (y - X\beta)^T (y - X\beta)$$

If X^TX is nonsingular, there is a unique closed-form solution:

$$\beta = (X^TX)^{-1} X^T y$$

This is the same idea as the simple least-squares formulas, generalized to any number of predictors.

How to find an approximated function

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \ldots + \beta_p X_p + \varepsilon$$

In the advertising example, we can consider the model with all three media:

$$\text{sales} = \beta_0 + \beta_1 \text{TV} + \beta_2 \text{radio} + \beta_3 \text{newspaper} + \varepsilon$$

Instead of one predictor, we now weigh several inputs at once — each β captures the effect of its own predictor, holding the others fixed.

Estimation & prediction

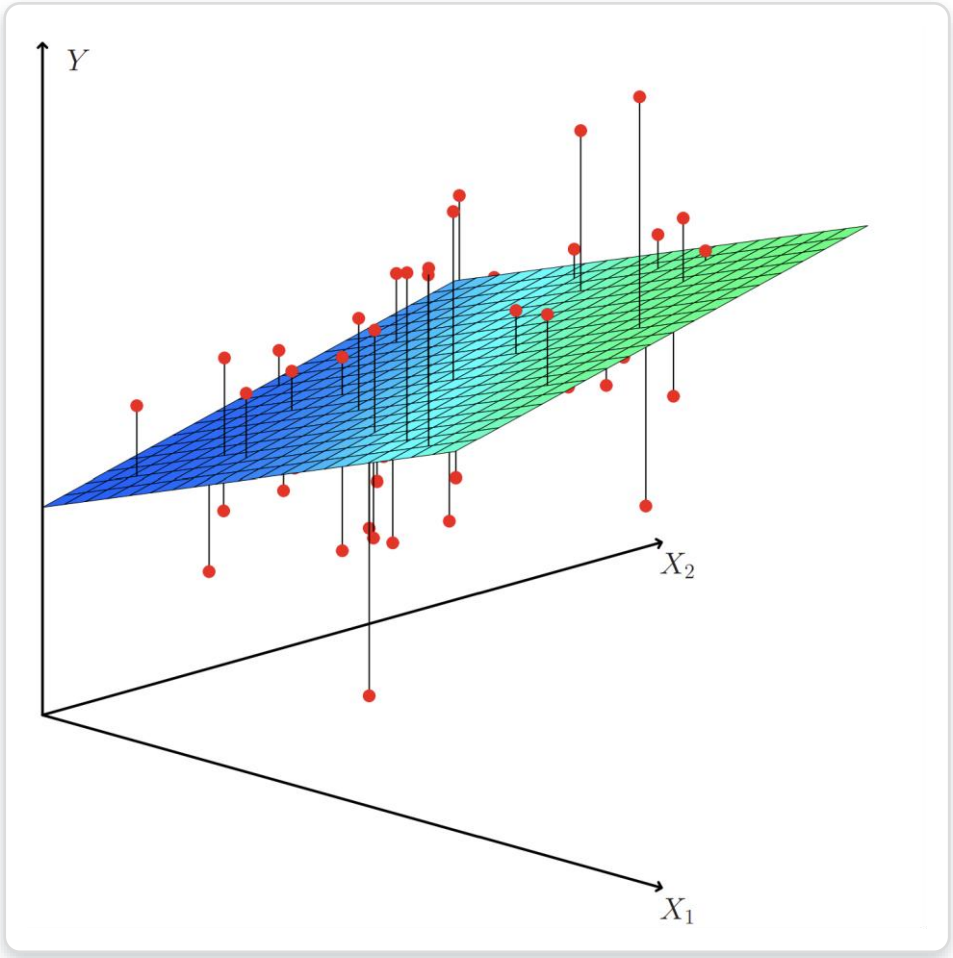
$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p$$

*We **estimate $\beta_0, \beta_1, \ldots, \beta_p$ that minimize the sum of squared residuals**:*

$$RSS = \Sigma (y_i - \beta_0 - \beta_1 x_{i1} - \ldots - \beta_p x_{ip})^2$$

In this three-dimensional setting (two predictors, one response), the least-squares regression line becomes a plane, chosen to minimize the sum of squared vertical distances between each observation and the plane.

The regression plane



OLS: sales ~ TV + radio + newspaper

PYTHON

```
est = smf.ols('sales ~ TV + radio + newspaper',
              advertising).fit()
est.summary()
```

	coef	std err	t	P>|t|
Intercept	2.9389	0.312	9.422	0.000
TV	0.0458	0.001	32.809	0.000
radio	0.1885	0.009	21.893	0.000
newspaper	-0.0010	0.006	-0.177	0.860

$R^2 = 0.8972$ (F = 570.3) — TV and radio are strongly significant; **newspaper is not** (p = 0.860, far above 0.05).

THEORY

Potential problems

When we fit a linear regression model to a dataset, several issues may occur:

1. Non-linearity of the response-predictor relationships
2. Correlation of error terms
3. Non-constant variance of error terms
4. Outliers
5. High-leverage points
6. Collinearity

Correlation amongst predictors

PYTHON

```
advertising.corr()
```

	TV	radio	newspaper	sales
TV	1.000	0.055	0.057	0.782
radio	0.055	1.000	0.354	0.576
newspaper	0.057	0.354	1.000	0.228
sales	0.782	0.576	0.228	1.000

Note:
- TV and sales are strongly correlated (0.782), radio moderately (0.576).
- radio and newspaper are also somewhat correlated (0.354) this partly explains why newspaper loses significance once radio is in the model….

Two-predictor model: radio + TV

PYTHON

```
regr = skl_lm.LinearRegression()

X = advertising[['radio','TV']].to_numpy()
y = advertising.sales

regr.fit(X, y)

print(regr.coef_)
print(regr.intercept_)
```

RESULT

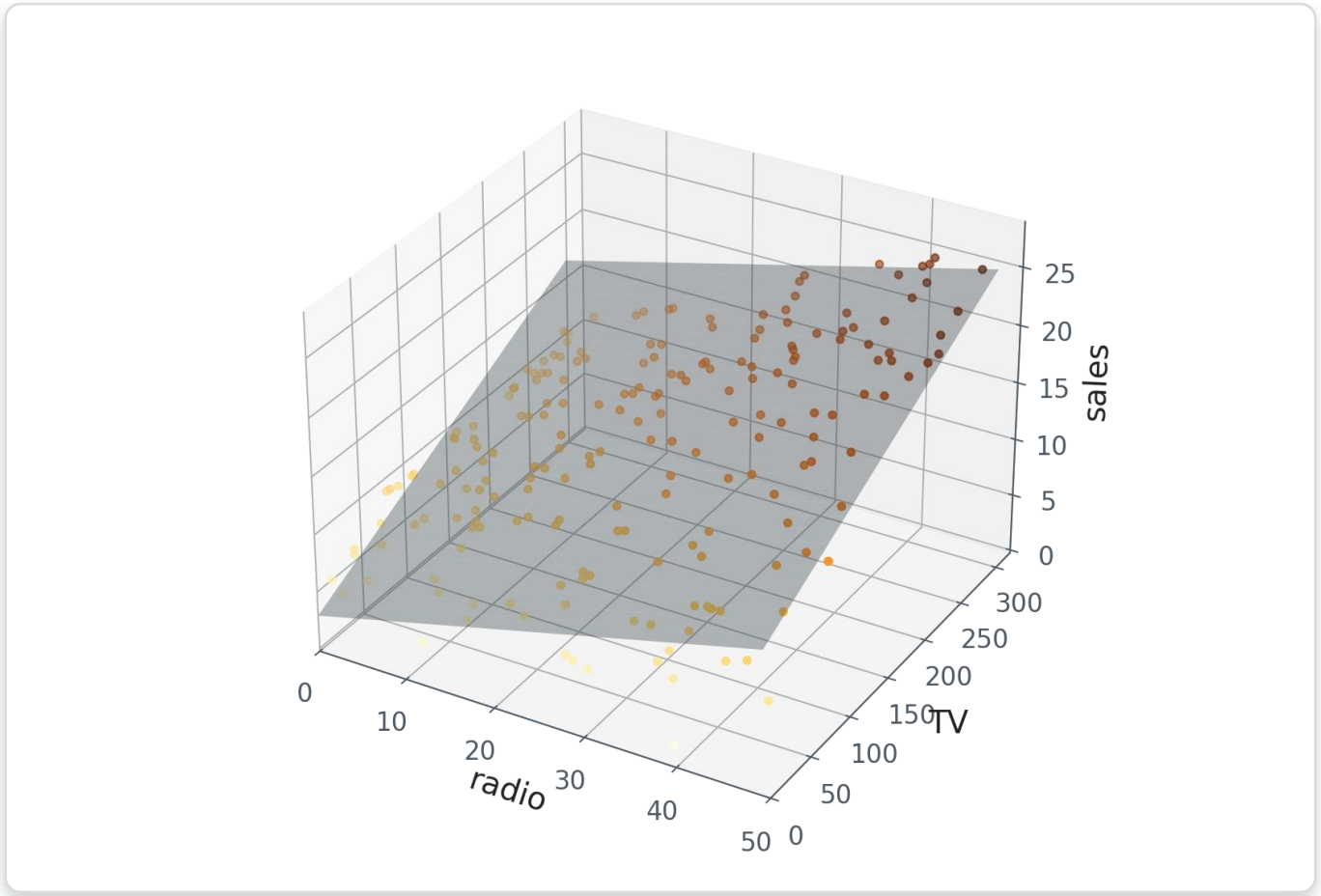
```
coef      = [0.1880, 0.0458]
              (radio,  TV)
intercept = 2.9211
```

These coefficients set up the plane we'll visualize next :
- sales rising with both radio and TV spend,
- radio having the steeper per-unit effect.

	radio	TV
mean	23.26	147.04
std	14.85	85.85
min	0.00	0.70
max	49.60	296.40

Visualizing the regression plane

sales predicted from radio and TV jointly: the plane is the model; the dots are the actual observations.



THEORY

Questions a regression model should answer

1. Which predictors X_1, X_2, ..., X_n are useful in predicting the response?

2. Are all the predictors useful, or only a subset?

3. How well does the model fit the data?

4. Given a set of predictor values, what response should we predict? and **how accurate is that prediction**?

Extending the linear model

- A simple way to accommodate non-linear relationships: polynomial regression
- Example: mpg (fuel efficiency) vs. horsepower, in the Auto dataset
- There is a pronounced, but clearly non-linear, relationship
- Fix: include transformed versions of the predictor, e.g. a quadratic term

$$\text{mpg} = \beta_0 + \beta_1 \text{horsepower} + \beta_2 \text{horsepower}^2 + \varepsilon$$

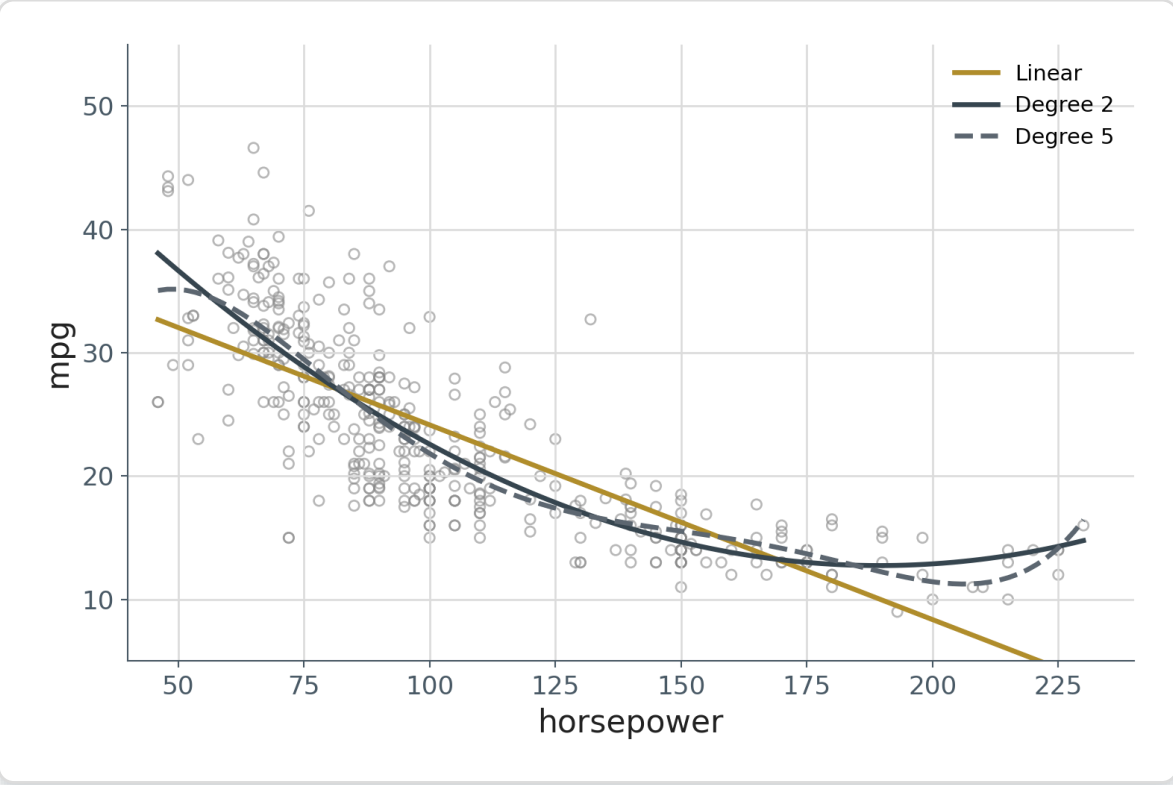
mpg vs. horsepower — which fit is best?

PYTHON

```
auto = pd.read_csv('Auto.csv',
    na_values='?').dropna()

sns.regplot(x="horsepower", y="mpg",
    data=auto, order=1, label='Linear')
sns.regplot(x="horsepower", y="mpg",
    data=auto, order=2, label='Degree 2')
sns.regplot(x="horsepower", y="mpg",
    data=auto, order=5, label='Degree 5')
```

392 observations (5 with missing horsepower dropped).



The quadratic model, fitted

PYTHON

```
auto['horsepower2'] = auto.horsepower**2

est = smf.ols('mpg ~ horsepower + horsepower2',
              auto).fit()
est.summary().tables[1]
```

	coef	std err	t	P>|t|
Intercept	56.9001	1.800	31.604	0.000
horsepower	-0.4662	0.031	-14.978	0.000
horsepower²	0.0012	0.000	10.080	0.000

$R^2 = 0.6876$ — both horsepower and horsepower² are highly significant ($p < 0.001$), confirming the curved relationship suggested by the chart.

FURTHER RESOURCES

Where to learn more

A few video walkthroughs that cover the same material from a different angle:

ISLP

- Simple Linear Regression
- Multiple Linear Regression
- Other Considerations in the Regression Model

StatQuest

- Linear Regression, Clearly Explained
- Multiple Regression, Clearly Explained

KEY TAKEAWAYS

Wrapping up

- Simple linear regression: $Y = \beta_0 + \beta_1 X + \varepsilon$, fit by minimizing the RSS (least squares)
- R^2 measures the proportion of variance explained (scale-free, between 0 and 1)
- Multiple regression extends this to several predictors: $\beta = (X^T X)^{-1} X^T y$
- Watch for collinearity between predictors! it can make a variable look non-significant
- Polynomial terms let a linear model capture curved (non-linear) relationships